

- [CPSC 375: High-Performance Computing](#)

[CPSC 375: High-Performance Computing](#)

Spring 2026

- [Syllabus](#)
- [Schedule](#)
- [Upload](#)
- [The Cluster Project](#)

Homework 12

NOTE: You are not required to hand in the following exercises, but you are strongly encouraged to complete them to strengthen your understanding of the concepts covered in class.

- For each of the following scenarios, identify which of the four classes of thread-unsafe functions is being described:
 - A function that utilizes a static internal counter to track how many times it has been called across the entire program execution.
 - A function that calls `gethostbyname` to resolve a URL.
 - A function that calculates a value and returns a pointer to a static struct containing the results.
 - A function that increments a global variable without using a semaphore.
- Explain the specific condition that makes a function reentrant compared to just thread-safe.
- If you have a Class 2 thread-unsafe function (relying on persistent state), describe the standard fix required to make that function reentrant.
- The `pthread_create` function requires a pointer to a start routine. Write the exact function signature required for this start routine.
- Explain why the `arg` parameter in `pthread_create` is of type `void *` and how a programmer can use this to pass multiple distinct values to a single thread.
- The Collatz Conjecture [L. Collatz, 1937] states that any sequence of natural numbers, $x_0, x_1, \dots$, where $x_{i+1} = x_i/2$ if x_i is even; $x_{i+1} = 3x_i + 1$ if x_i is odd, will eventually reach 1. Write a C program (`collatz.c`) which will experimentally verify the conjecture for all positive integers $x < N$ for some large N . If the conjecture is verified for all x , your program should print:

Conjecture verified for all positive integers $< N$
Total number of steps: $nSteps$

If the conjecture would not hold for some x (you will be famous if you find one such number), your program should print:

Found a counterexample: x

- Use long integers throughout the program. Make sure the value of N is passed to the program at the command-line. Use the `time()` function to measure running time.
7. Rewrite `collatz.c` using T threads. Call your program `collatz_threaded.c`. Make sure the values of N and T are passed to the program at the command-line. Compare its running time with the sequential version using different values of N and T . Include the timing results in the header comment.

- **Welcome: Sean**

- [LogOut](#)

